SIMATS SCHOOL OF ENGINEERING

Approved By AICTE | IET-UK Accreditation

Course Code: DSA03	Course Name: Natural Language Processing
Branch: CSE	Year : 2026

Name: R.Rohith Manikanta
Reg no: 192425326

Assignment 2

1. Figure 4.1 shows the bigram counts from a piece of a bigram grammar from the Berkeley Restaurant Project. Note that the majority of the values are zero. Perform the add-1 smoothed counts for the below bigram probability values. (10M) (CO2) (BLT4)

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Figure 4.1 Bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences.

2. Suppose someone took all the words in a sentence and reordered them randomly. Write a program which take as input bag of words such a bag of words and produces as output a guess at the original order. You will need to an N-gram grammar produced by your N-gram program (on some corpus. This task is sometimes called bag generation. (10M) (CO2) (BLT4)

3. Use the Penn Treebank tagset to tag each word in the following sentences from Damon Runyon's short stories. You may ignore punctuation. Some of these are quite difficult; do your best. (10M) (CO2) (BLT4)

   a. It is a nice night.

   b. This crap game is over a garage in Fifty-second Street. . .

   c. . . .Nobody ever takes the newspapers she sells . . .

   d. He is a tall, skinny guy with a long, sad, mean-looking kisser, and a mournful voice. e. . . . I am sitting in Mindy's restaurant putting on the gefillte fish, which is a dish I am very fond of, . . .

   f. When a guy and a doll get to taking peeks back and forth at each other, why there you are indeed.

Answer 1. Add-1 (Laplace) smoothed bigram counts and probabilities

Add-1 smoothing adds one to every bigram count, so the zero entries in Figure 4.1 stop being zero. With vocabulary size V = 1446, the smoothed probability and the smoothed (reconstituted) count are

$$P^*(w_n \mid w_{n-1}) = (C(w_{n-1}\, w_n) + 1) / (C(w_{n-1}) + V)$$

$$C^*(w_{n-1}\, w_n) = (C(w_{n-1}\, w_n) + 1) \times C(w_{n-1}) / (C(w_{n-1}) + V)$$

The unigram counts needed for the denominators come from the same Berkeley Restaurant corpus (they are not printed in Figure 4.1): C(i)=2533, C(want)=927, C(to)=2417, C(eat)=746, C(chinese)=158, C(food)=1093, C(lunch)=341, C(spend)=278.

Worked examples. C*(i want) = (827+1) × 2533 / (2533+1446) = 828 × 2533 / 3979 = **527.1**; P*(want | i) = 828 / 3979 = **0.2081**. For a zero count, C*(i to) = 1 × 2533 / 3979 = **0.64** and P*(to | i) = 1 / 3979 = **0.00025**.

Table 1. Add-1 smoothed bigram counts C* (rows = previous word, columns = next word)

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527.1	0.6	6.4	0.6	0.6	0.6	1.9
want	1.2	0.4	237.9	0.8	2.7	2.7	2.3	0.8
to	1.9	0.6	3.1	429.8	1.9	0.6	4.4	132.6
eat	0.3	0.3	1.0	0.3	5.8	1.0	14.6	0.3
chinese	0.2	0.1	0.1	0.1	0.1	8.2	0.2	0.1
food	6.9	0.4	6.9	0.4	0.9	2.2	0.4	0.4
lunch	0.6	0.2	0.2	0.2	0.2	0.4	0.2	0.2
spend	0.3	0.2	0.3	0.2	0.2	0.2	0.2	0.2

Table 2. Add-1 smoothed bigram probabilities P*(column | row)

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.2081	0.0003	0.0025	0.0003	0.0003	0.0003	0.0008
want	0.0013	0.0004	0.2566	0.0008	0.0029	0.0029	0.0025	0.0008
to	0.0008	0.0003	0.0013	0.1778	0.0008	0.0003	0.0018	0.0549
eat	0.0005	0.0005	0.0014	0.0005	0.0078	0.0014	0.0196	0.0005
chinese	0.0012	0.0006	0.0006	0.0006	0.0006	0.0517	0.0012	0.0006
food	0.0063	0.0004	0.0063	0.0004	0.0008	0.0020	0.0004	0.0004
lunch	0.0017	0.0006	0.0006	0.0006	0.0006	0.0011	0.0006	0.0006
spend	0.0012	0.0006	0.0012	0.0006	0.0006	0.0006	0.0006	0.0006

Smoothing moves probability mass from the seen bigrams to the unseen ones: C*(i want) falls from 827 to 527.1 and C*(to eat) from 686 to 429.8, while every former zero rises to a small positive value, so no bigram has probability 0 any more.

Answer 2. Bag generation with a bigram grammar (Python)

Approach. The program first builds a bigram language model with add-1 smoothing from a training corpus (sentence markers <s> and </s> are added). Given a shuffled bag of words, it searches for the ordering with the highest probability $P(w_1|\text{<s>}) \cdot P(w_2|w_1) \cdots P(\text{</s>}|w_n)$. Because n words have n! orderings, it uses a beam search: it builds the sentence left to right, at each step tries every unused word, and keeps only the best 200 partial sentences. Log-probabilities are summed to avoid underflow. The corpus below is a small demonstration corpus; any larger text file can be used by passing its lines to Bigram().

```
import itertools, math
from collections import Counter

CORPUS = """i want to eat chinese food
i want to eat lunch
i want to spend money
i want chinese food
to eat lunch is nice
i want to eat
tell me about chinese food
i would like to eat lunch
i want food
give me chinese food
i want to spend lunch
want to eat chinese food
i am looking for chinese food
i want to eat food
list all chinese food
chinese food is good
i want to spend more money
we want to eat lunch today""".split("\n")

class Bigram:
    def __init__(self, sentences):
        self.uni, self.bi = Counter(), Counter()
        for s in sentences:
            w = ["<s>"] + s.lower().split() + ["</s>"]
            self.uni.update(w)
            self.bi.update(zip(w, w[1:]))
        self.V = len(self.uni)
    def logp(self, a, b):            # add-1 smoothed log P(b | a)
        return math.log((self.bi[(a, b)] + 1) / (self.uni[a] + self.V))
    def score(self, seq):
        w = ["<s>"] + list(seq) + ["</s>"]
        return sum(self.logp(a, b) for a, b in zip(w, w[1:]))

def order(bag, lm, beam=200):
    """Guess the original order of a bag of words (beam search over bigram scores)."""
    bag = [w.lower() for w in bag]
    beams = [(0.0, ["<s>"], tuple(sorted(bag)))]
    for _ in bag:
        nxt = {}
        for sc, seq, rest in beams:
            for w in set(rest):
                r = list(rest); r.remove(w)
                key = (tuple(seq + [w]), tuple(r))
                nxt[key] = (sc + lm.logp(seq[-1], w), seq + [w], tuple(r))
        beams = sorted(nxt.values(), key=lambda x: -x[0])[:beam]
    best = max(beams, key=lambda b: b[0] + lm.logp(b[1][-1], "</s>"))
    return " ".join(best[1][1:])

if __name__ == "__main__":
    lm = Bigram(CORPUS)
```

```
 for bag in (["food", "want", "i", "chinese"],
             ["eat", "to", "lunch", "want", "i"],
             ["spend", "i", "money", "to", "want"],
             ["is", "good", "food", "chinese"]):
     print(sorted(bag), "->", order(bag, lm))
```

Output obtained by running the program:

```
['chinese', 'food', 'i', 'want'] -> i want chinese food
['eat', 'i', 'lunch', 'to', 'want'] -> i want to eat lunch
['i', 'money', 'spend', 'to', 'want'] -> i want to spend money
['chinese', 'food', 'good', 'is'] -> is good chinese food
```

The first three bags are restored exactly. The fourth comes out as “is good chinese food” instead of “chinese food is good”, because a bigram model only sees one word of context and the tiny corpus offers little evidence. A larger corpus, a trigram model (change the context to the last two words) or a larger beam would improve the guesses.

Answer 3. Penn Treebank POS tags (punctuation ignored)

Each word is written as word/TAG. Possessive ’s is split off as its own token (POS), as in the Treebank.

a. It/PRP is/VBZ a/DT nice/JJ night/NN

b. This/DT crap/NN game/NN is/VBZ over/IN a/DT garage/NN in/IN Fifty-second/JJ Street/NNP

c. Nobody/NN ever/RB takes/VBZ the/DT newspapers/NNS she/PRP sells/VBZ

d. He/PRP is/VBZ a/DT tall/JJ skinny/JJ guy/NN with/IN a/DT long/JJ sad/JJ mean-looking/JJ kisser/NN and/CC a/DT mournful/JJ voice/NN

e. I/PRP am/VBP sitting/VBG in/IN Mindy/NNP ’s/POS restaurant/NN putting/VBG on/RP the/DT gefillte/JJ fish/NN which/WDT is/VBZ a/DT dish/NN I/PRP am/VBP very/RB fond/JJ of/IN

f. When/WRB a/DT guy/NN and/CC a/DT doll/NN get/VBP to/TO taking/VBG peeks/NNS back/RB and/CC forth/RB at/IN each/DT other/JJ why/UH there/RB you/PRP are/VBP indeed/RB

Notes on the harder choices: “Nobody” is tagged NN (the Treebank treats it as a noun-like indefinite). “crap” modifies “game” so it is a noun (NN) used as a modifier. “Fifty-second” is an ordinal adjective (JJ) and “Street” belongs to a proper name (NNP). “newspapers she sells” is a reduced relative clause, so “she” is PRP and “sells” VBZ. “putting on” is a phrasal verb with “on” as a particle (RP), not a preposition. “gefillte” is a foreign-origin modifier tagged JJ, and “which” is a wh-determiner/pronoun (WDT). “of” stays IN even though it is stranded at the end of the clause. In (f), “When” is WRB, “get to taking” is VBP + TO + VBG, “back and forth” are adverbs (RB), and “why” is an interjection here (UH), which is debatable and could also be tagged WRB.